Linear Regression For Fun And Profit

By Evan Miller

October 19, 2012 (Changes)

Let's say you run a retail website and you're trying to decide which picture to use on the home page: a dog or a cat.

You run an A/B test, and the dog picture is the clear winner. That is, on average, people bought more stuff after seeing the dog picture than the cat picture. So take down the cat picture and call it a day, right?

If you do that, you're potentially alienating the cat lovers who come to your website, even though they form a minority. A better approach would be to try to identify the cat people and show them the cat picture, and show everybody else the dog picture.

Major websites spend a lot of money and hire a lot of Ph.D.'s to solve the problem of on-the-fly personalization, with a branch of arcane mathematics called "machine learning". Fortunately, you don't have to be Amazon.com or the Terminator to take advantage of machine-learning techniques. If you're already running A/B tests, you probably already have more than enough data to create a first-rate personalization service, and in this blog post I'll give you absolutely all the mathematics you need to implement such a service on the most humble of hardware.

We're going to use a well-worn method called *multiple linear regression*. It may not be the best model for all occasions, but it's a good starting point even for complex applications, and it's surprisingly straightforward to implement for simple ones. It's so simple, in fact, that we won't need to use specialized statistical packages, just some simple code and a call or two to your system's linear-algebra library. If equations scare you, don't fret: if you can understand Pivot Tables in Excel, I promise you can understand linear regression. But I'm going to go fast, so hold on tight!

Background

The single most important equation in all of applied statistics is this:

$$\beta = (X'X)^{-1}X'Y$$

This is the $F = ma$ of multivariate analysis, discovered by Carl Friedrich Gauss in 1794. It's an equation that tells us how to estimate a linear relationship between a variable of interest Y (say, sales) and everything else X (say, the user's web browser, operating system, anything else we can think of). If we plug some data into this equation, we can calculate β then use it to predict the level of Y for some values of X:

$$y = x\beta$$

That is, once we've estimated β, we can predict how much stuff someone will buy (y) based on their observable characteristics (x). Note that x and β are vectors, so multiplying them together is really just multiplying the individual components and adding up the results.

So here is the strategy: we'll set up a random experiment and estimate a β for the cat-picture group, and a separate β for the dog-picture group, and then use the results to predict whether a given user will likely buy more stuff after being shown the cat picture or the dog picture.

Let's take a look at that equation again:

$$\beta = (X'X)^{-1}X'Y$$

It looks very scary, with matrix transposes, inversions, and multiplications, but it's really four simple operations which you can perform in (almost) any programming language.

Step 1: Compute $X'X$.

They won't tell you this in graduate school, but $X'X$ is really just making a Pivot Table where you make a row and column for each category value, and in each cell, add up the number of people who have the pair of values corresponding to the row and column. Here's an example table that I just made up, where I made a category for each operating system and each web browser.

	Windows	Mac	Linux	Chrome	Firefox	Internet Explorer	Safari	Total
Windows	2819	0	0	344	459	1405	611	2819
Mac	0	1482	0	349	502	0	631	1482
Linux	0	0	345	200	145	0	0	345
Chrome	344	349	200	893	0	0	0	893
Firefox	459	502	145	0	1106	0	0	1106
Internet Explorer	1405	0	0	0	0	1405	0	1405
Safari	611	631	0	0	0	0	1242	1242
Total	2819	1482	345	893	1106	1405	1242	4646

There are a lot of zeroes here, because (for our purposes) you can't be both a Windows user and a Mac user, or a Safari user and a Chrome user.

Next (this is important), you need to delete a row and column for each category variable to avoid a pitfall called the dummy trap. Don't worry, "dummy" is referring to the variable, not to the analyst. Here I'll delete "Windows" and "Internet Explorer", so the result looks like this:

	Mac	Linux	Chrome	Firefox	Safari	Total
Mac	1482	0	349	502	631	1482
Linux	0	345	200	145	0	345
Chrome	349	200	893	0	0	893
Firefox	502	145	0	1106	0	1106
Safari	631	0	0	0	1242	1242
Total	1482	345	893	1106	1242	4646

That table is exactly the $X'X$ matrix. You probably thought we'd be transposing and multiplying matrices to arrive at it, but deep down it's just a Pivot Table missing a couple of rows and columns. Note that the "Total" column here is crucial, so be sure to keep it around for the next step.

Step 2: Invert the matrix from Step 1 to form $(X'X)^{-1}$

This is the hardest part of the whole procedure, and you should probably use a numerical library to do it. If you're using C or have access to C, I recommend LAPACK. Here's the code I use on Mac OS X, boilerplate included:

```
#include <Accelerate/Accelerate.h> /* or your local LAPACK library */

void invert_matrix(__CLPK_doublereal *matrix, __CLPK_integer k) {
    __CLPK_integer ispec = 1; /* optimal block size */
    __CLPK_integer info; /* Error information */
    __CLPK_integer dgetri_block_size = ilaenv_(&ispec, "DGETRI", "", NULL, NULL, NULL, NULL,
                                               sizeof("DGETRI")-1, sizeof("")-1);
    __CLPK_integer iwork_dim = k * dgetri_block_size;
    __CLPK_integer *ipiv = malloc(k * k * sizeof(__CLPK_integer));
    __CLPK_doublereal *iwork = malloc(iwork_dim * sizeof(__CLPK_doublereal));
    dgetrf_(&k, &k, matrix, &k, ipiv, &info);
    dgetri_(&k, matrix, &k, ipiv, iwork, &iwork_dim, &info);
    free(ipiv);
    free(iwork);
}
```

If you want to understand exactly how that code works, check out the LAPACK documentation for `dgetrf` and `dgetri`. Anyway all you need to know is that that function will invert a square matrix of dimension k, using technical wizardry that I do not fully comprehend. Note that the running time of matrix inversion is $O(k^3)$; this is not a problem for small matrices like the one here, but could slow you down once you have thousands of columns.

Anyway, once you've inverted your matrix and patted yourself on the back, you can move on to Step 3. We have $(X'X)^{-1}$ in hand, so let's compute $X'Y$.

Step 3: Compute $X'Y$

This took me three years to figure out, but $X'Y$ is just summing up Y by all values of X, e.g., sales by observable characteristics:

Windows	$20,943
Mac	$14,094
Linux	$3,059
Chrome	$5,928
Firefox	$8,294
Internet Explorer	$19,590
Safari	$4,284
Total	$38,096

Note that "Total" here means the total across all users, not across all categories (i.e. no double-counting). As before, we need to delete a row for each category variable to avoid the dummy trap, so that whittles our table down to:

Mac	$14,094
Linux	$3,059
Chrome	$5,928
Firefox	$8,294
Safari	$4,284
Total	$38,096

Piece of cake! Again, no matrix transposes or multiplications needed — this is just a Pivot Table missing a couple of rows. Let's move on to Step 4.

Step 4: Multiply the result from Step 2 by the result from Step 3 to form β

Vector-matrix multiplication sounds fancy but it's just high-school math. It's just a matter of taking a dot-product of the $X'Y$ vector from Step 3 with each row in the inverted matrix from Step 2. Here is some C code that will do the trick:

```
void multiply(const double *matrix_from_step2,
              const double *vector_from_step3,
              size_t k, double *beta) {
   int i, j;
   for (i=0; i<k; i++) {
       beta[i] = 0.0;
       for (j=0; j<k; j++) {
           beta[i] += matrix_from_step2[i*k+j] * vector_from_step3[j];
       }
    }
}
```

...and we're done! Because the matrix from Step 2 is symmetric, it doesn't matter whether you've stored your data in "column-major" or "row-major" order — the above code will work either way.

We've now multiplied $(X'X)^{-1}$ by $(X'Y)$ and stored the resulting vector. That is to say, we've calculated β. That wasn't so hard, now was it?

The β vector has a straightforward interpretation: each entry tells you the effect of a particular characteristic on the outcome, *after taking into account all other characteristics in the regression*. It consists of a "Mac effect", a "Linux effect", a "Firefox effect", and so on. The best part is that we never have to worry whether an "Internet Explorer effect" is really the result of a "Windows effect" (or vice versa), because the linear regression sorts all that out for us. That's the magic of linear regression.

Anyway, the last entry in β, corresponding to "Total", gives us the baseline value. Note that the columns we deleted from our Pivot Tables are actually the baseline characteristics. So for example, the estimated "Mac effect" would be relative to a Windows user, the estimated "Firefox effect" would be relative to an Internet Explorer user, and so on.

Now if you do this computation for each branch of a randomized experiment, you'll get several beta vectors, e.g.

$$\beta_{cat} = (X'X)^{-1}_{cat} X'Y_{cat}$$

$$\beta_{dog} = (X'X)^{-1}_{dog} X'Y_{dog}$$

Then when someone new comes to your site, you just add up the β values for that user's characteristics, once for each branch, and pick the branch with the highest predicted value. Here's an example calculation with made-up numbers:

	User	β_{cat}	β_{dog}	Cat prediction	Dog prediction
Mac	1	$5.84	-$2.38	$5.84	-$2.38
Linux	0	$0.92	$0.44	$0.00	$0.00
Chrome	0	-$4.58	$1.32	$0.00	$0.00
Firefox	1	-$1.43	$1.75	-$1.43	$1.75
Safari	0	$2.12	$1.84	$0.00	$0.00
Baseline	1	$80.23	$83.28	$80.23	$83.28
Total				$84.64	$82.65

For the user in this example (Mac user, Firefox), the predicted sales for the cat picture is higher ($84.64 instead of $82.65), so we'll show them that instead of the dog picture, and use the expected profits to buy ourselves an iPhone app.

The example I've given here is simple, but you can load up the regression with as many category variables as you think are relevant (provided they are not collinear). Keep in mind that as you add more columns to the model, the matrix inversion becomes more costly, to the tune of $O(k^3)$. But you only have to calculate β once per branch of the experiment, so in practice it probably won't be a problem. And once β is in hand, making each prediction requires only $O(k)$ multiplications and additions.

The only time you'd want to completely reject a branch of the experiment (say, the cat picture) is if it could never beat another branch for any feasible set of values. It's easy to figure out if this is the case; just take the difference between the two β vectors, and sum up the highest (positive) value for each category variable in the resulting vector, as well as the lowest (negative) value for each category variable. Add the baseline entry of the difference vector to each of these sums. If the "best-case" sum and the "worst-case" sum are both positive or both negative, you should completely reject the branch of the experiment with the lower baseline value; otherwise, you should keep them both and serve them selectively.

If you're wondering about whether the estimates from the two groups differ significantly in a statistical sense, it's possible to do a Chow test on the two β vectors. To compute the p-value you'll need access to the integral of an F distribution. I recommend finding a statistical library or package to compute the p-value for you, but enterprising programmers can roll their own function using TOMS 708 and ASA 063.

Conclusion

Linear regression is a powerful technique that lets you make a prediction from many input variables. For simple applications like this, the math is completely straightforward, and if you have a matrix inversion function handy, it is easy to implement. Although computing β is a relatively expensive operation, you have to perform the computation only once for each branch at the end of the experiment. If you care about the statistical significance of your predictors, the statistical properties of linear regression are well-known, and the standard errors are simple to calculate.

So the next time you finish running a A/B test, instead of declaring A or B to be the universal, unquestioned, and eternal victor, you might consider linear regression as a way to continue showing A to some users and B to other users, profiting from the data you collected during the experiment. As you've seen here, predicting outcomes from the estimated parameters only requires adding up a few numbers, and so the technique is quite suitable for the real-time personalization of Internet applications.

Changes

- Nov. 2, 2012 – Add information about the Chow test.

You're reading evanmiller.org, a random collection of math, tech, and musings. If you liked this you might also enjoy:

- *How Not To Run an A/B Test*
- *How Not To Sort By Average Rating*
- *How To Read an Unlabeled Sales Chart*

Get new articles as they're published, via LinkedIn, Twitter, or RSS.

Want to look for statistical patterns in your MySQL, PostgreSQL, or SQLite database? My desktop statistics software ***Wizard*** *can help you analyze* ***more data in less time*** *and* ***communicate discoveries visually*** *without spending days struggling with pointless command syntax. Check it out!*

Wizard
Statistics the Mac way

Back to Evan Miller's home page – Subscribe to RSS – LinkedIn – Twitter

---